

Igor André Pegoraro Santana

**Aplicando o Processo de Mineração de Dados
para Aquisição de Informação Contextual para
Sistemas de Recomendação Sensíveis ao
Contexto**

Maringá

20 de Novembro de 2017

Igor André Pegoraro Santana

Aplicando o Processo de Mineração de Dados para Aquisição de Informação Contextual para Sistemas de Recomendação Sensíveis ao Contexto

Trabalho monográfico acadêmico apresentado como avaliação periódica à disciplina 5210 - Trabalho de Conclusão de Curso para o curso de Bacharelado em Informática.

Universidade Estadual de Maringá

Centro de Tecnologia

Departamento de Informática

Orientador: Prof. Dr. Marcos Aurélio Domingues

Maringá

20 de Novembro de 2017

Agradecimentos

Resumo

Para a utilização de sistemas de recomendação sensíveis ao contexto, vê-se necessário a descoberta de informações contextuais referentes aos dados que serão utilizados por tais sistemas. Devido a falta de métodos para a obtenção automática de informações contextuais com dados georreferenciados, como por exemplo latitude e longitude, neste trabalho foi proposto o uso do processo de mineração de dados junto com a técnica de agrupamento para a obtenção automática dessas informações. Essas técnicas de agrupamento foram então comparadas e, a partir dos resultados, foi feita uma análise da utilização das informações em um sistema de recomendação sensível ao contexto

Palavras-chaves: Sistemas de Recomendação. Sistemas de Recomendação Sensíveis ao Contexto. Técnicas de Agrupamento. Mineração de Dados.

Abstract

To use a context-aware recommender system, it's necessary the discovery of contextual information from the data that will be used on it. Because there is not enough methods to get these contextual informations with georeferenced data, this work will propose the use of the data mining process together with the clustering technique to automatically get these information. Different clustering techniques that use georeferenced data will be compared in order to obtain an improvement in the result of the recommender system.

Key-words: Recommender System. Context Aware Recommender System. Clustering Techniques. Data Mining.

Lista de ilustrações

Figura 1 – Etapas do processo de Mineração de Dados	12
Figura 2 – Exemplos de dados em seus respectivos <i>clusters</i>	17
Figura 3 – Classificação de sistemas de recomendação sensíveis ao contexto	26
Figura 4 – Processo de aprendizagem usando a abordagem DaVI	28
Figura 5 – Exemplos da visão do banco de dados por meio de gráfico	30
Figura 6 – Exemplo do algoritmo DBSCAN	31
Figura 7 – Funcionamento do algoritmo KMeans	33
Figura 8 – Exemplo do algoritmo aglomerativo	34
Figura 9 – Exemplo de utilização do algoritmo DaVI-Best	37
Figura 10 – F-measure dos algoritmos de agrupamento	48

Lista de tabelas

Tabela 1	– Exemplo dos atributos da base de dados.	42
Tabela 2	– Bases de dados após o pré-processamento	42
Tabela 3	– Exemplo das regiões atribuídas para cada item.	43
Tabela 4	– Resultados obtidos com o ItemKNN no Foursquare	45
Tabela 5	– Resultados obtidos com o ItemKNN no Yelp	46
Tabela 6	– Resultados obtidos com o BPR no Foursquare	47
Tabela 7	– Resultados obtidos com o BPR no Yelp	47

Sumário

	INTRODUÇÃO	9
I	FUNDAMENTAÇÃO TEÓRICA	11
1	MINERAÇÃO DE DADOS	12
1.1	Conhecimento de Domínio	13
1.2	Pré-processamento	13
1.3	Extração de Padrões	15
1.4	Pós-processamento	15
1.5	Utilização do Conhecimento	16
2	TÉCNICAS DE AGRUPAMENTO	17
3	SISTEMAS DE RECOMENDAÇÃO	20
3.1	Algoritmos de filtragem colaborativa	21
3.2	Algoritmos de filtragem baseada em conteúdo	22
3.3	Algoritmos de filtragem híbridos	23
3.4	Sistemas de recomendação sensíveis ao contexto	24
II	ALGORITMOS UTILIZADOS	29
4	ALGORITMOS DE AGRUPAMENTO	30
4.1	Algoritmo DBSCAN	30
4.2	Algoritmo KMeans	31
4.3	Algoritmo Aglomerativo	34
5	ALGORITMOS DE RECOMENDAÇÃO	35
5.1	Algoritmo ItemKNN	35
5.2	Algoritmo BPR	36
5.3	Algoritmo DaVI-Best	37
III	TRABALHO DESENVOLVIDO	39
6	MINERAÇÃO DE DADOS	40
6.1	Conhecimento do Domínio	40
6.2	Pré-processamento	40

6.2.1	Foursquare	40
6.2.2	Yelp	41
6.2.3	Transformação dos Dados	41
6.3	Extração de Padrões	42
6.4	Utilização do Conhecimento	43
7	RESULTADOS OBTIDOS	45
7.1	ItemKNN	45
7.2	BPR	46
7.3	Discussão dos resultados	48
	Conclusão	49
	REFERÊNCIAS	50

INTRODUÇÃO

A cobertura e o volume de conjuntos de dados georreferenciados crescem extensivamente a cada dia, em razão da utilização de dispositivos de coleta geográfica juntamente com tecnologias de localização, como o Sistema de Posicionamento Global, do inglês *Global Positioning System* (GPS), que permite pesquisadores obterem uma grande quantidade de dados. Dispositivos de monitoramento de meio ambiente, por exemplo, proporcionam uma vasta quantidade de dados como imagens, vídeos e sons. Os métodos de análise espacial tradicional, no entanto, foram projetados para a utilização com conjuntos de dados homogêneos, pequenos e com amostragem científica, tornando essas técnicas insuficientes na utilização de dados de aplicações reais, em consequência do grande volume de dados e da natureza heterogênea dos mesmos (MILLER, 2008).

A Descoberta de Conhecimento em Banco de Dados, do inglês *Knowledge Discovery in Databases* (KDD), é uma análise exploratória automática de grandes repositórios de dados. O KDD é um processo organizado de identificação de padrões válidos, novos e úteis em conjunto de dados complexos. A Mineração de Dados é a essência de um processo KDD, pois envolve o uso de algoritmos que exploram os dados, criam o modelo e descobrem padrões (MAIMON; ROKACH, 2010).

Em Fayyad, Piatetsky-Shapiro e Smyt (1996), são citadas algumas áreas no qual a descoberta de conhecimento em banco de dados pode ser útil:

- **Marketing.** Em marketing, a principal aplicação é analisar bancos de dados de usuários para identificar grupos de clientes e prever seu comportamento. Uma outra aplicação é fazer uma análise de sistemas de “cesta de mercado”, que tentam achar padrões do tipo “Se um cliente comprou X, é também provável que ele compre Y e Z.”;
- **Detecção de Fraude.** Sistemas que buscam fraudes em transações financeiras utilizam o processo de mineração de dados para encontrar padrões que possam indicar lavagem de dinheiro e demais fraudes.

Uma área que é beneficiada com a Mineração de Dados é a área de sistemas de personalização, como por exemplo os sistemas de recomendação, que utilizam de informações do comportamento do usuário em aplicações para fornecer um conteúdo específico para o mesmo.

Sistemas de Recomendação coletam informações das preferências dos usuários para um conjunto de itens, como por exemplo filmes, livros e músicas. Essa informação pode

ser adquirida de maneira explícita (por meio de avaliações de usuários) ou implícita (monitorando o comportamento do usuário). Um sistema de recomendação pode utilizar informações demográficas dos usuários (idade, nacionalidade, gênero), informações sociais (publicações, seguidores) e também informações originárias de sistemas embarcados (localizações de GPS, informações de saúde, etc) (BOBADILLA et al., 2013).

O uso primário de sistemas de recomendação é direcionado a usuários que não possuem experiência para avaliar a excessiva quantidade de itens que uma aplicação possa oferecer. Um exemplo desse uso se dá em um sistema de recomendação de livros que dá assistência aos usuários para escolher um livro para ler. Em sua forma mais simples, as recomendações personalizadas para cada usuário são oferecidas como uma lista ordenada de itens. Para construir essa lista, um sistema de recomendação tenta prever quais são os produtos ou serviços mais adequados baseado nas preferências do usuário (RICCI; ROKACH; SHAPIRA, 2011).

A motivação desse trabalho está na utilização das informações georreferenciadas obtidas no processo de mineração de dados como contexto para sistemas de recomendação sensíveis ao contexto, ao invés da utilização de sistemas de recomendação tradicionais que não utilizariam esta informação., visando

Esse trabalho tem como objetivo realizar um processo de mineração de dados em duas bases de dados que possuem informações georreferenciadas, com o propósito de separar esses dados em grupos por meio de técnicas de agrupamento. Esses grupos serão avaliados utilizando sistemas de recomendação sensíveis ao contexto, visando descobrir quais técnicas de agrupamento resultam em resultados melhores para sistemas de recomendação.

No [Capítulo 1](#), serão apresentados conceitos e fundamentos sobre mineração de dados, que serão aplicados durante o desenvolvimento do trabalho. No [Capítulo 2](#), serão explicados conceitos relativos as técnicas de agrupamento que serão avaliadas. Os conceitos de sistemas de recomendação apresentados no [Capítulo 3](#) servirão como base para os algoritmos utilizados no trabalho.

No [Capítulo 4](#) e [Capítulo 5](#), são apresentados os algoritmos que foram utilizados durante o desenvolvimento do trabalho, sendo eles os de agrupamento e os de recomendação. O [Capítulo 6](#) descreve os passos do processo de mineração de dados feito no trabalho, enquanto no [Capítulo 7](#) são divulgados os resultados obtidos nos experimentos.

Parte I

Fundamentação Teórica

1 Mineração de Dados

O processo de Mineração de Dados surge da necessidade de análise e extração de conhecimento em bases de dados que, com o passar do tempo, foram crescendo de tamanho, tornando assim então muito difícil a tarefa de uma análise manual dos dados.

De acordo com [Rezende et al. \(2003\)](#), o objetivo principal do processo de Extração de Conhecimento de Bases de Dados é encontrar conhecimento útil e que pode ser utilizado como auxílio em tomada de decisão a partir de um conjunto de dados. O processo compreende também uma área multidisciplinar composta de técnicas de Base de Dados, Estatística e Inteligência Artificial.

A definição mais aceita de Mineração de dados vem de [Fayyad, Piatetsky-Shapiro e Smyth \(1996\)](#): “Extração de Conhecimento de Base de Dados é o processo de identificação de padrões válidos, novos, potencialmente úteis e compreensíveis embutidos nos dados”. O processo de mineração de dados, como proposto em [Rezende et al. \(2003\)](#), é dividido em cinco etapas, como ilustrado na 1: Conhecimento de Domínio, Pré-processamento, Extração de Padrões, Pós-processamento e Utilização do Conhecimento.

Figura 1 – Etapas do processo de Mineração de Dados



Fonte – [Rezende et al. \(2003\)](#)

1.1 Conhecimento de Domínio

A etapa de Conhecimento de Domínio consiste em estudar o domínio de aplicação dos dados, assim como definir o objetivo que o usuário final deseja alcançar após o processo. De acordo com [Rezende et al. \(2003\)](#), algumas questões devem ser respondidas na etapa de conhecimento de domínio:

- Quais são as principais metas do processo?
- Quais critérios de desempenho são importantes?
- O conhecimento extraído deve ser compreensível a seres humanos ou um modelo do tipo caixa-preta é apropriado?
- Qual deve ser a relação entre simplicidade e precisão do conhecimento extraído?

A partir das respostas a essas perguntas, a compreensão do domínio do problema e do resultado desejado na etapa final pode guiar o processo de mineração de dados, durante todas as outras etapas. Um bom estudo do domínio pode facilitar na etapa de pré-processamento, por exemplo, para garantir que dados importantes não sejam removidos ou aplicando uma medida de peso em algum atributo que possua uma importância maior.

1.2 Pré-processamento

A etapa de Pré-processamento, por vezes chamada de **Preparação de Dados**, é comumente a parte do processo que é mais custosa computacionalmente e a mais demorada. Durante essa etapa, várias tarefas podem ser realizadas, como a limpeza dos dados originais, integração de dados de outras fontes e a transformação dos dados para um modo que possa ser usado como entrada na próxima etapa ([MOBASHER, 2011](#)).

A aplicação do processo de mineração de dados de uma maneira efetiva para aplicações *Web* depende da utilização correta de métodos de pré-processamento. A fonte primária de informação dessas aplicações são arquivos de *log*, que podem ser de acessos a um servidor *web* ou da aplicação em si. Estes dados podem ser categorizados em quatro grupos, como visto em [Mobasher \(2011\)](#):

1. **Dados de Uso.** Estes dados são coletados automaticamente e representam o comportamento do consumidor com relação a navegação do mesmo na aplicação. Cada vez que um acesso é realizado na aplicação e uma chamada é feita para o servidor, é gerado uma entrada nos *logs* de acesso.
2. **Dados de Conteúdo.** Os dados de conteúdo são a coleção de objetos que são enviados aos usuários, como por exemplo arquivos HTML, arquivos de áudio e en-

tradas de banco de dados. Junto desses dados, podem estar inclusos metadados relacionados ao conteúdo.

3. **Dados de Estrutura.** Estes dados são representados pela organização de conteúdo dentro da aplicação, como por exemplo a estrutura HTML ou a ligação entre páginas por meio de *hyperlinks*, dentro de uma aplicação *Web*.
4. **Dados do Usuário.** Dados adicionais dos usuários como informações de perfil, avaliações de objetos, compras realizadas ou histórico de visitas compõe os dados do usuário. Estes dados podem ser capturados de maneira anônima ou com consentimento do usuário.

Uma fusão desses dados pode ser feita como uma tarefa da etapa de pré-processamento, para que as demais tarefas possam ser executadas (MOBASHER, 2011). Em Rezende et al. (2003), são definidas tarefas importantes a serem realizadas, como a limpeza e a seleção e redução dos dados. Durante a tarefa de limpeza, o objetivo é garantir a qualidade do conjunto e remover dados que possuam problemas advindos da coleta dos mesmos e que possam atrapalhar durante a extração de padrões, por exemplo, erros de digitação e atributos vazios. A seleção e redução dos dados é utilizada em bases de dados que possam ter um conjunto muito grande de exemplos e de atributos e, por isso, possuem um custo computacional muito alto. Como visto em WEISS; INDURKHYA apud Rezende et al. (2003), existem três maneiras nas quais essa redução pode ser feita:

1. **Redução do número de exemplos.** A abordagem mais utilizada para a redução de exemplos é a amostragem aleatória, devido a sua tendência de produzir amostras representativas;
2. **Redução do número de atributos.** Esta redução tem como finalidade reduzir o espaço de busca pela solução, reduzindo o número de atributos e encontrando assim um subconjunto que não tenha impacto na qualidade da solução final;
3. **Redução do número de valores de um atributo.** Consiste em aplicar técnicas afim de modificar os valores de um atributo, como por exemplo a discretização, que consiste em agrupar os valores de atributos, transformando atributos contínuos em atributos discretos.

O pré-processamento pode também incluir a tarefa de remoção de acessos únicos da base de dados. Por diversos motivos, um usuário pode utilizar o serviço da aplicação apenas uma vez, gerando então acessos únicos na base de dados. Estes acessos únicos, quando utilizados por sistemas de recomendação, resultam em uma avaliação do sistema com resultados inferiores (DOMINGUES; SOARES; JORGE, 2008).

1.3 Extração de Padrões

O objetivo da Extração de Padrões é executar as tarefas de mineração de dados que serão utilizadas para obter os padrões de interesse definido na etapa de Conhecimento de Domínio. Conforme [Rezende et al. \(2003\)](#), “Nessa etapa é realizada a escolha, a configuração e execução de um ou mais algoritmos para extração de conhecimento. Como esta etapa é um processo iterativo, pode ser necessário que seja executada diversas vezes para ajustar o conjunto de parâmetros visando à obtenção de resultados mais adequados aos objetivos preestabelecidos”.

As duas principais tarefas da Mineração de Dados são predição e descrição. A predição envolve usar variáveis com respostas conhecidas no banco de dados para prever valores futuros ou desconhecidos de outras variáveis de interesse, enquanto a descrição tem seu foco em descobrir padrões escondidos nos dados ([FAYYAD; PIATETSKY-SHAPIRO; SMYT, 1996](#)).

A escolha dos algoritmos que serão utilizados durante esta etapa depende inteiramente da tarefa que será escolhida para o problema, pois cada uma tem resultados diferentes e que são adequados para diferentes objetivos. Em uma tarefa de predição, por exemplo, podem ser usadas Árvore de Decisão e Redes Bayesianas. Os algoritmos ainda devem ser ajustados de acordo com seus parâmetros visando obter os melhores resultados possíveis ([REZENDE et al., 2003](#)).

1.4 Pós-processamento

Após a extração de dados, a etapa de pós-processamento é responsável por definir quais padrões serão utilizados em uma tomada de decisão. Segundo [Liu e Hsu \(1996\)](#), algumas perguntas devem ser respondidas com relação aos padrões descobertos:

1. Os padrões representam o conhecimento prévio ao processo?
2. Caso não, de que maneira esse conhecimento difere?
3. Qual parte do conhecimento prévio está correto, com relação aos padrões descobertos?

Caso o número de padrões seja demasiadamente grande para que os usuários finais consigam analisar manualmente, devem ser usadas métricas para garantir que os padrões obtidos possuam um desempenho adequado, até como um meio de filtragem para padrões com desempenho baixo. Em [FLACH; LAVRAC; ZUPAN](#) apud [Rezende et al. \(2003\)](#), são citadas algumas medidas de desempenhos que podem ser utilizadas, que são: precisão, erro,

confiança negativa, sensibilidade, especificidade, cobertura, suporte, satisfação, velocidade e tempo de aprendizado.

Caso o conhecimento obtido e analisado na fase de pós-processamento não esteja de acordo com o interesse do usuário, o processo de mineração de dados, por ser iterativo, pode ser repetido alterando técnicas e parâmetros visando obter resultados diferentes e melhores dos que já foram obtidos.

1.5 Utilização do Conhecimento

O conhecimento obtido através do processo de mineração de dados pode ajudar organizações a determinar o valor de seus clientes, desenvolver estratégias de *marketing* de produtos e serviços, avaliar a efetividade de campanhas promocionais e fornecer um conteúdo personalizado para seus usuários (LIU, 2011).

No [Capítulo 2](#), são descritas as técnicas que são comumente usadas no processo de mineração de dados para *Web* e que foram aplicadas no trabalho, que são as técnicas de agrupamento. Ao utilizar essas técnicas, grupos de usuários com padrões similares são estabelecidos e, com esse conhecimento, pode ser realizada uma segmentação de mercado em aplicações de *e-commerce* ou uma personalização de conteúdo *Web* para os usuários que possuem interesses similares (MOBASHER, 2011).

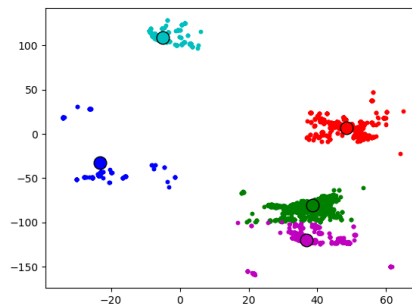
par Uma abordagem para a utilização do resultado obtido no processo de mineração de dados é realizar uma integração com um sistema de personalização, como por exemplo um sistema de recomendação, de modo que esses resultados atuem como entrada no sistema de personalização. Esse sistema de personalização irá então adaptar o conteúdo exibido para o usuário de acordo com esse conhecimento que foi obtido analisando interações passadas desse mesmo usuário (PIERRAKOS et al., 2003).

2 Técnicas de Agrupamento

As técnicas de agrupamento são utilizadas como tarefa descritiva durante a etapa de Extração de Padrões, ou seja, são utilizadas para encontrar padrões dentro de um conjunto de dados.

De acordo com [Liu \(2011\)](#), agrupamento compreende o processo de organizar os objetos de uma base de dados em *clusters* que possuem similaridades entre si. Para que estes dados sejam organizados, é necessário uma função de similaridade para determinar o quão similares dois objetos são. Os algoritmos que implementam técnicas de agrupamento são divididos em dois grupos: hierárquicos e particional. A [Figura 2](#) apresenta *clusters* que foram encontrados em um conjunto de dados, onde cada cor representa um *cluster* diferente e o ponto maior de cada *cluster* representa seu ponto principal.

Figura 2 – Exemplos de dados em seus respectivos *clusters*.



Conforme visto em [Han e Kamber \(2006\)](#), o grupo de algoritmos hierárquicos corresponde a algoritmos que criam uma divisão de dados hierárquicos a partir de um conjunto de dados, de duas formas: aglomerativa ou divisiva. Os métodos aglomerativos tratam, no começo de um processo iterativo, cada objeto como sendo um *cluster* e, durante as iterações, os objetos mais próximos se fundirão em um *cluster* maior, até que todos os objetos estejam em um *cluster* único.

Na forma divisiva, é formado um *cluster* com todos os objetos e, nas sucessivas iterações, divisões são realizadas até que todos os objetos sejam um *cluster* ou até que alguma condição de parada seja alcançada. Esses algoritmos hierárquicos possuem uma característica que, quando uma divisão (ou junção) de um *cluster* ocorre, a mesma não pode ser desfeita, sendo esta uma importante técnica para evitar conexões errôneas.

Para determinar a distância entre dois *clusters*, [Rokach \(2010\)](#) introduz três métodos:

1. **Single-link clustering.** Estes métodos se utilizam da menor distância entre quaisquer pontos de um *cluster* para o outro, para então definir a distância entre estes *clusters*.
2. **Complete-link clustering.** A distância entre dois *clusters* é igual a maior distância entre os membros de um *cluster* para o outro.
3. **Average-link clustering.** Este método utiliza-se da média entre as distâncias de pontos de um *cluster* para outro para calcular a distância entre ambos.

Algoritmos de particionamento criam uma partição inicial dos dados em *clusters* e, utilizando técnicas iterativas de relocação, tentam melhorar essas mesmas partições trocando objetos entre os *clusters*. Esses algoritmos são definidos em Han e Kamber (2006) como: “Dado um banco de dados de n objetos, um método de particionamento constrói k partições dos dados, sendo que cada partição representa um *cluster* e $k \leq n$ ”. Conforme a definição, podemos dizer que cada *cluster* possui pelo menos um objeto e que cada objeto está em pelo menos um *cluster*.

O algoritmo *k-means*, como apresentado em Liu (2011), é o algoritmo mais conhecido com relação a algoritmos particionais de *clustering*. Ele atribui k centros de *clusters*, que são chamados de centroides, e após a definição desses, os outros objetos são atribuídos ao centroide mais próximo, que são recalculados usando os objetos atuais do *cluster*. A execução do algoritmo só é interrompida de acordo com o critério de parada definido, que pode ser um dos seguintes:

1. Não-atribuição de objetos em *clusters* diferentes;
2. Não ocorrer mudança de centroides.

Para definir quais objetos estão “próximos” uns dos outros, um algoritmo de agrupamento necessita de uma função que calculará a distância entre esses objetos. Estas funções de distância diferem de acordo com o tipo do atributo que ela deve calcular a distância. Uma função de distância de um atributo numérico, por exemplo, é diferente de uma função de distância de um atributo nominal. Em Liu (2011), os atributos são relacionados com funções de distância da seguinte forma:

- **Atributos numéricos.** As funções utilizadas para calcular a distância são um caso especial da função **Distância de Minkowski**, representada por:

$$\text{dist}(x_i, x_j) = (|x_{i1} - x_{j1}|^h + |x_{i2} - x_{j2}|^h + \dots + |x_{in} - x_{jn}|^h)^{\frac{1}{h}},$$

onde $\text{dist}(x_i, x_j)$ representa a distância entre os objetos x_i e x_j e h é um inteiro positivo. Caso $h = 2$, a função é chamada de **Distância Euclidiana**,

$$\text{dist}(x_i, x_j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{in} - x_{jn})^2}.$$

Caso $h = 1$, a função é chamada de **Distância Manhattan**,

$$\text{dist}(x_i, x_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{in} - x_{jn}|.$$

- **Atributos binários.** Existem dois tipos de atributos binários a serem considerados neste caso, atributos simétricos ou assimétricos. Um atributo é simétrico se seus dois estados (0 ou 1) possuem a mesma importância, enquanto que em um atributo assimétrico um estado tem o peso maior que o de outro. Dado que:

- a : número de atributos em que ambos os objetos tem valor verdadeiro;
- b : número de atributos que possuem valor verdadeiro em x_i e falso em x_j ;
- c : número de atributos que possuem valor verdadeiro em x_j e falso em x_i ;
- d : número de atributos em que ambos os objetos tem valor falso.

A função responsável pelo cálculo de atributos binários simétricos é a **distância de correspondência simples**:

$$\text{dist}(x_i, x_j) = \frac{b + c}{a + b + c + d},$$

e para atributos binários assimétricos, a função utilizada é chamada de **distância Jaccard**:

$$\text{dist}(x_i, x_j) = \frac{b + c}{a + b + c}.$$

- **Atributos nominais.** Os atributos nominais possuem uma grande similaridade com os atributos binários na hora de calcular a distância entre dois objetos, utilizando a função de **distância de correspondência simples**. Dado dois objetos x_i e x_j , seja o número de atributos r e o número de valores que são iguais em ambos objetos seja q , a função de distância é definida como:

$$\text{dist}(x_i, x_j) = \frac{r - q}{r}.$$

Quando os objetos possuem diversos tipos de atributos, se torna necessário a utilização de algum método para calcular a distância deste para com outros objetos. Um possível método é identificar algum tipo de atributo que aparece mais vezes no objeto e transformar os demais tipos para este atributo dominante. Além deste método, pode ser computada a distância de cada atributo dos dois objetos e então somá-las para chegar a uma distância final (LIU, 2011).

No [Capítulo 4](#), são descritos os algoritmos de agrupamento que foram utilizados durante o desenvolvimento do trabalho.

3 Sistemas de Recomendação

Um sistema de recomendação é uma tecnologia de filtragem de informação que pode ser utilizada para prever avaliações de itens e/ou gerar um *ranking* de itens que podem ser do interesse do usuário. “Item” é um termo usado para denotar o que um sistema de recomendação recomendaria para um usuário, que normalmente tem um foco específico em um tipo de item, como por exemplo filmes e livros (RICCI; ROKACH; SHAPIRA, 2011). Como recomendações geralmente são personalizadas, diferentes usuários ou grupos de usuários podem receber sugestões diversas (ADOMAVICIUS; TUZHILIN, 2011).

Em Adomavicius e Tuzhilin (2011), são descritos vários motivos que demonstram o porquê do investimento feito em sistemas de recomendação, como por exemplo:

- **Aumentar o número de itens vendidos.** Como um sistema de recomendação é capaz de recomendar itens personalizados ao usuário, estes itens possuem uma probabilidade maior de atender as vontades do usuário. Do ponto de vista de um prestador de serviços, a meta principal é aumentar o número de usuários que aceitam recomendações e utilizam estes itens recomendados;
- **Melhor entendimento das preferências de um usuário.** Um sistema de recomendação coleta informações sobre as preferências dos usuários da aplicação, possibilitando a utilização desses dados em outras áreas, como remover itens que não estão sendo vendidos e aumentar a produção de itens que são mais vendidos;
- **Vender itens diversos.** Em uma aplicação com um número grande de itens, o usuário pode não conseguir visualizar todos os seus itens que são de seu interesse. Um sistema de recomendação pode recomendar itens que não estão entre os mais visualizados para o usuário, baseado em suas preferências.

Em Adomavicius e Tuzhilin (2005), um Sistema de Recomendação tem a seguinte definição: Seja C o conjunto de todos os usuários e S o conjunto de todos os itens que podem ser recomendados. Seja u a função de utilidade que medirá o quão útil o item s é para o usuário c , ou seja, $u : C \times S \rightarrow R$, sendo R um conjunto ordenado de avaliações. Então, para cada usuário $c \in C$, queremos escolher um item $s' \in S$ que maximizará a utilidade do item para um usuário que, em um sistema de recomendação, representa uma avaliação que indica o quanto um usuário gostou de um item. Formalmente, é descrito como:

$$\forall c \in C, s'_c = \arg \max_{s \in S} u(c, s).$$

Existem três tipos de técnicas que são utilizadas em sistemas de recomendação, que são as seguintes: filtragem colaborativa, filtragem baseada em conteúdo e abordagens híbridas. Além dessas técnicas, na próxima seção serão apresentados também conceitos relacionados a sistemas de recomendação sensíveis ao contexto.

3.1 Algoritmos de filtragem colaborativa

Algoritmos de filtragem colaborativa fazem recomendações para um usuário tendo como referência recomendações feitas por usuários parecidos, por meio de uma função de similaridade. Em [Adomavicius e Tuzhilin \(2005\)](#), os autores apresentam a seguinte definição: “A utilidade $u(c, s)$ de um item s para um usuário c é estimada baseada nas utilidades $u(c_j, s)$, atribuídas ao item s pelos usuários $c_j \in C$ que são “similares” ao usuário c ”. Em [BREESE; HECKERMAN; KADIE](#) apud [Ansari, Essegaier e Kohli \(2000\)](#) os algoritmos de filtragem colaborativa são divididos em dois grupos: algoritmos baseados em memória (ou algoritmos baseados em heurísticas) e algoritmos baseados em modelo.

Os algoritmos baseados em memória utilizam heurísticas para calcular as avaliações de um usuário com base nos itens já avaliados por outros usuários. O valor de uma avaliação desconhecida $r_{c,s}$, de acordo com [Adomavicius e Tuzhilin \(2005\)](#), para um usuário c e um item s é calculada por meio de uma agregação das avaliações de outros usuários para o mesmo item s :

$$r_{c,s} = \text{aggr}_{c' \in \hat{C}} r_{c',s},$$

onde $\hat{C}$ representa o conjunto de usuários similares ao usuário c que avaliaram um item s .

Enquanto os algoritmos baseados em memória utilizam heurísticas, os algoritmos baseados em modelo usam as avaliações conhecidas para aprender modelos preditivos. De acordo com [Adomavicius e Tuzhilin \(2011\)](#), “sua ideia geral é modelar as interações entre usuário e item com fatores representando características latentes dos usuários e dos itens no sistema de recomendação. Este modelo então é treinado usando os dados disponíveis e usado posteriormente para prever avaliações de usuários para novos itens”.

O algoritmo *k-Nearest Neighbour* (kNN) é visto como referência dos algoritmos de filtragem coletiva e possui duas versões: baseado em usuários e baseado em itens. Caso o processo de recomendação seja baseado em usuários, o algoritmo kNN teria a seguinte forma:

1. Determinar os k usuários mais similares ao usuário u ;
2. Fazer uma agregação das avaliações feitas pelos usuários similares para os itens não recomendados ao usuário u ;

3. Extrair as predições realizadas no passo anterior e escolher as N melhores para o usuário u .

Como visto em [Bobadilla et al. \(2013\)](#), o algoritmo kNN tem como suas principais virtudes a sua simplicidade, junto com boas recomendações. Um problema que o algoritmo kNN enfrenta é a escalabilidade, que ocorre quando o tamanho do banco de dados utilizado contém uma quantidade muito grande de usuários, que torna o processo de encontrar usuários similares lento. Uma maneira de reduzir este problema é utilizar a versão baseada em itens, visto que os itens mais parecidos podem ser guardados e então utilizados para fazer as recomendações, já que a similaridade entre itens é mais estável do que a similaridade entre usuários, já que a quantidade de itens cresce menos do que a quantidade de usuários.

3.2 Algoritmos de filtragem baseada em conteúdo

Os algoritmos baseados em conteúdo aprendem a recomendar itens que são similares a itens que já foram avaliados no passado pelo mesmo usuário ([ADOMAVICIUS; TUZHILIN, 2011](#)). Se um usuário avaliar positivamente itens que possuem palavras-chave como “futebol” e “basquete”, um algoritmo baseado em conteúdo poderá recomendar itens que estão relacionados a esportes. Um perfil do usuário é criado de acordo com a utilização da aplicação, com informações relativas a interesses do mesmo. Em [Pazzani e Billsus \(2007\)](#), são definidos dois tipos de informações que podem ser atribuídas a um usuário:

1. Um modelo da preferência do usuário, como por exemplo a descrição do tipo dos itens no qual o usuário possui interesse.
2. Um histórico das interações do usuário para com o sistema de recomendação. Este tipo de informação inclui tanto os itens que o usuário visualizou, quanto informações de compra de produtos ou a avaliação que ele deu para tal objeto.

Os sistemas baseados em conteúdo vem ganhando importância nas redes sociais, pois conteúdos relacionados a comentários, opiniões, compartilhamentos e *tags* estão sendo adicionados como atributos de itens, possibilitando o uso dessas informações durante o processo de recomendação. Três passos são definidos para um algoritmo baseado em conteúdo ([BOBADILLA et al., 2013](#)):

1. Extrair os atributos dos itens para fazer a recomendação;
2. Comparar os atributos desses itens com as preferências dos usuários ativos;
3. Recomendar itens que possuem características que reflitam os interesses do usuário.

Criar o modelo das preferências de um usuário pode ser visto como um problema de classificação, onde os dados de treino são categorizados nas duas categorias: “itens que o usuário gosta” e “itens que o usuário não gosta”. Estes dados que serão categorizados podem ser recolhidos por meio de *feedback* explícito do usuário ou de uma observação implícita da interação do usuário com a aplicação (PAZZANI; BILLSUS, 2007).

A utilização de sistemas de recomendação baseados em conteúdo possui diversas vantagens, como visto em Lops, Gemmis e Semeraro (2011):

- **Independência do Usuário.** Como os sistemas de recomendação baseado em conteúdo fazem suas predições considerando apenas o perfil do usuário, o que possibilita uma aplicação com poucos usuários conseguir fazer boas recomendações;
- **Transparência.** Como as recomendações são feitas levando em conta as características do item e do usuário, é possível descrever quais características foram levadas em consideração para que o item tenha sido recomendado ao usuário final;
- **Item Recente.** Como o algoritmo não depende da avaliação de outros usuários parecidos em algum item específico, itens que foram adicionados recentemente podem ser recomendados ao usuário devido a similaridade de características.

Ainda que o sistema de recomendação baseado em conteúdo possua diversas vantagens, alguns problemas inerentes a sua natureza de recomendação podem ser observados. Ainda em Lops, Gemmis e Semeraro (2011), as seguintes desvantagens são descritas:

- **Análise limitada de conteúdo.** É necessário existir conhecimento prévio do conteúdo no qual o sistema de recomendação será aplicado, visto que um sistema de recomendação baseado em conteúdo não consegue fornecer sugestões boas se o conteúdo analisado não contém informação suficiente para decidir se o usuário gosta ou não do item;
- **Super especialização.** Este efeito ocorre pois um sistema de recomendação baseado em conteúdo tende, com o tempo, a recomendar itens sem nenhum grau de novidade para o usuário, limitando o número de itens que podem ser recomendados;
- **Usuário recente.** Um usuário, para obter recomendações boas, precisa ter avaliado outros itens previamente para que um modelo sobre suas preferências seja criado.

3.3 Algoritmos de filtragem híbridos

Algoritmos híbridos buscam combinar os pontos fortes dos algoritmos de filtragem colaborativa e baseado em conteúdo. Em Adomavicius e Tuzhilin (2005) são descritos métodos para combinar os algoritmos:

1. Combinar as predições do algoritmo colaborativo e do baseado em conteúdo. Isto pode ser feito comparando as predições para descobrir qual tem o maior grau de confiança e então, usá-la;
2. Incorporar características de algoritmos baseado em conteúdo em um algoritmo de filtragem colaborativa por meio das características do usuário obtidas pelo algoritmo baseado em conteúdo para fazer a predição em vez de utilizar apenas os itens avaliados. Isto pode fazer com que um item seja recomendado não só quando possui uma avaliação alta de outros usuários, mas também quando possuir características semelhantes a do usuário;
3. Incorporar características de algoritmos de filtragem colaborativa em um algoritmo baseado em conteúdo;
4. Construir um modelo unificado que incorpora característica de ambos algoritmos.

Utilizar um algoritmo híbrido pode aliviar problemas que a abordagem colaborativa e outros sistemas de recomendação podem ter. Um exemplo de problema a ser evitado é o *cold-start*, que ocorre em sistemas de recomendação baseados em conteúdo.

O problema de *cold-start* corresponde ao problema de novo usuário, que ocorre quando o usuário não avaliou itens suficientes para o sistema de recomendação entender, e o problema de novo item, que ocorre quando um item não foi avaliado por um número considerável de usuários e, portanto, o sistema de recomendação não seria capaz de recomendá-lo (ADOMAVICIUS et al., 2005).

3.4 Sistemas de recomendação sensíveis ao contexto

Um sistema de recomendação sensível ao contexto se utiliza de informações contextuais que também são utilizadas para auxiliar na predição de avaliações de itens, como o horário no qual o item foi acessado, se o usuário estava acompanhado por alguém, entre outras. Conforme Adomavicius e Tuzhilin (2005), “a predição exata das preferências do consumidor depende, sem dúvida, do grau em que incorporamos a informação contextual relevante no método de recomendação”.

O contexto, como visto em Adomavicius e Tuzhilin (2011), pode ser definido por meio de uma visão diferente em cada área de aplicação. Portanto, muitas definições podem ser encontradas dentro de cada área, ou até dentro de subárea. A seguir, são apresentados exemplos dessas definições:

- **Mineração de Dados.** O contexto, como encontrado na literatura de mineração de dados, foi definido em (BERRY; LINOFF, 1997 apud PALMISANO; TUZHILIN;

(GORGOGLIONE, 2008) como eventos que caracterizam a vida de um usuário e pode determinar uma mudança nas suas preferências, *status* e valor para uma empresa. Exemplo dessa definição são um trabalho novo, casamento, nascimento de um filho, entre outros;

- **Customização de *e-commerce*.** De acordo com Palmisano, Tuzhilin e Gorgoglione (2008), a intenção da compra por parte de um consumidor em um *e-commerce*, determina uma informação contextual. Como um exemplo disso, um usuário pode comprar um livro para aprender sobre determinado conteúdo ou ferramentas para utilizar em um *hobby*;
- **Recuperação de Conhecimento.** O contexto, quando utilizado em recuperação de informação, tem como foco a sua utilização em problemas de curto-prazo e interesses do usuário no momento específico, como por exemplo: "encontrar todos os restaurantes em Maringá que servem lanches vegetarianos"(PALMISANO; TUZHILIN; GORGOGLIONE, 2008).

Um sistema de recomendação sensível ao contexto pode ser descrito como um sistema multidimensional, no qual cada contexto representa uma dimensão. De uma maneira formal Adomavicius e Tuzhilin (2001), mostra que dadas as dimensões $D_1, D_2, \dots, D_n$, o espaço de recomendação será $S = D_1 \times D_2 \times \dots \times D_n$. Sendo *Ratings* o domínio representando o conjunto de todas as avaliações, então a função de recomendação R é definida como:

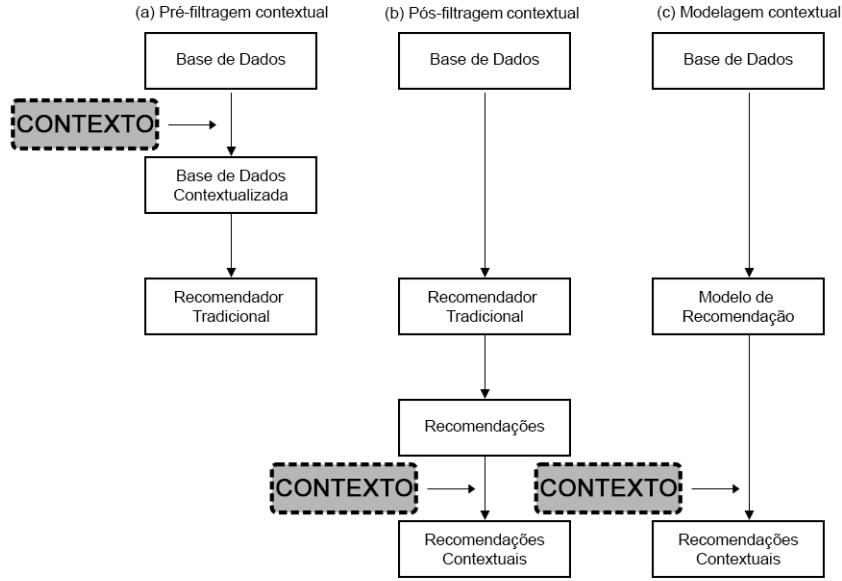
$$R : D_1 \times D_2 \times \dots \times D_n \rightarrow Ratings.$$

Em Adomavicius e Tuzhilin (2001), um exemplo é apresentado como uma aplicação para recomendação de filmes, que se utiliza do espaço de recomendação $Usuário \times Item \times Tempo$ para fazer as recomendações ao usuário. Após isto, uma função de avaliação deste espaço é utilizada para especificar o quanto um usuário $u \in Usuário$ gosta de um item $i \in Item$ no momento $t \in Tempo$, com a forma $R(u, i, t)$. Então, neste exemplo, a avaliação de um usuário João para o filme "O Vento Levou", que foi assistido durante a manhã com uma nota 5, teria o formato $R(João, O Vento Levou, Manhã) = 5$.

Em Adomavicius e Tuzhilin (2011), é feita uma subdivisão dos sistemas de recomendação sensíveis ao contexto em 3 grupos, **pré-filtragem contextual**, **pós-filtragem contextual** e **modelagem contextual**, como demonstrado na Figura 3.

O sistema de recomendação sensível ao contexto que se utiliza de uma abordagem de **pré-filtragem contextual** faz o uso da informação contextual já em uma primeira fase, na base de dados, para então utilizar um recomendador tradicional nestes dados com as informações de contexto.

Figura 3 – Classificação de sistemas de recomendação sensíveis ao contexto



Fonte – Imagem adaptada de [Adomavicius e Tuzhilin \(2011\)](#)

A abordagem *Reduction Based*, como apresentada em [Adomavicius et al. \(2005\)](#), é uma das técnicas de pré-filtragem contextual que tem como finalidade reduzir o problema de recomendação multidimensional para um problema bidimensional, fazendo com que possa ser utilizado qualquer sistema de recomendação tradicional.

Para demonstrar como a redução é feita, [Adomavicius et al. \(2005\)](#) propõe o seguinte exemplo. Assuma que

$$R_{U^{usuário} \times I^{item}}^D : U \times I \rightarrow Avaliação$$

é uma função de estimativa de avaliação bidimensional que, dadas as avaliações D , pode calcular uma predição para qualquer futura avaliação. Em uma função tridimensional, no entanto, é adicionada uma nova dimensão que será utilizada para fazer a predição. No caso desse exemplo, a dimensão utilizada será Tempo, definindo então a função tridimensional como

$$R_{U^{usuário} \times I^{item} \times T^{tempo}}^D : U \times I \times T \rightarrow Avaliação$$

onde D contém entradas no formato $\langle \text{usuário}, \text{item}, \text{tempo}, \text{avaliação} \rangle$ para as avaliações. Uma função de predição tridimensional pode ser expressa por meio de uma função bidimensional como

$$\forall (u, i, t) \in U \times I \times T, R_{U^{usuário} \times I^{item} \times T^{tempo}}^D(u, i, t) = R_{U^{usuário} \times I^{item}}^{D[Tempo=t]}(u, i)$$

onde $D[Tempo = t](U^{usuário}, I^{item}, Avaliação)$ denota o conjunto de avaliações obtidos de D selecionando apenas entradas onde a dimensão Tempo tem valor t .

Ainda em [Adomavicius et al. \(2005\)](#), a ideia da utilização não somente do contexto t na avaliação (u, i, t) , mas a utilização de um conjunto denominado **segmento contextual**. Como um exemplo deste segmento, em um valor de dimensão $t = Quarta\ Feira$ poderíamos utilizar um superconjunto $t = Dia\ De\ Semana$. A utilização do segmento visa resolver o problema da falta de avaliações que um valor de dimensão possa ter, agrupando-o então com valores similares. A função de recomendação, utilizando-se de segmentos, ficaria com a forma

$$R_{U_{usuário} \times Item \times Tempo}^D(u, i, t) = R_{U_{usuário} \times Item}^{D[Tempo \in S_t](Usuário, Item AGGR(avaliação))}(u, i)$$

onde $AGGR(avaliação)$ é utilizada pois pode haver valores t diferentes.

Esta redução dimensional pode ser estendido para um método de redução genérico, capaz de reduzir um espaço de recomendação n -dimensional para um espaço m -dimensional, em que $m < n$. De um modo geral, $m = 2$ devido aos sistemas de recomendação utilizarem apenas duas dimensões para fazerem suas recomendações ([ADOMAVICIUS et al., 2005](#)).

Para a **pós-filtragem contextual**, como visto na [Figura 3](#), aplica a filtragem contextual após a recomendação ter sido realizada por um sistema de recomendação tradicional. Em [Adomavicius e Tuzhilin \(2011\)](#), este método pode ser classificado de duas maneiras: baseado em heurísticas e baseado em modelo. Os métodos baseado em heurística pretendem encontrar atributos de um usuário em algum contexto, e então utilizar esses atributos para ajustar as recomendações, das seguintes formas:

- Filtrando itens que foram recomendados que não possuem essas características, ou
- Ordenando esses itens se baseando no número de características que possuem.

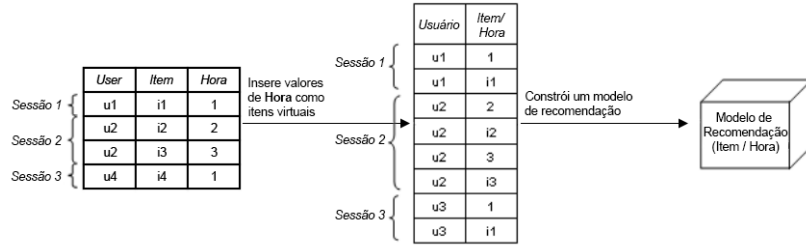
Para os métodos baseados em modelo, modelos preditivos são construídos para calcular a probabilidade no qual o usuário escolheria aquele item em um determinado contexto, usando então a probabilidade de relevância para ajustar as recomendações, das seguintes formas:

- Filtrando itens que tem uma probabilidade de relevância menor que um limite pré-definido, ou
- Ordenando esses itens se baseando na probabilidade de relevância que eles possuem.

Para a **modelagem contextual**, uma abordagem apresentada em [Domingues, Jorge e Soares \(2012\)](#) chamada DaVI (*Dimensions as Virtual Items*), trata dimensões adicionais como itens virtuais, que serão utilizados para criar o modelo de recomendação

junto com itens comuns, porém com a diferença de não serem recomendados para o usuário final.

Figura 4 – Processo de aprendizagem usando a abordagem DaVI



Fonte – Imagem adaptada de Domingues, Jorge e Soares (2012)

Como visto em Domingues, Jorge e Soares (2012), é definido p como o número de usuários $U = \{u_1, u_2, \dots, u_p\}$ e q o número de itens possíveis de serem recomendados $I = \{i_1, i_2, \dots, i_q\}$. Além disso, as dimensões (informações contextuais) $D = \{D_1, D_2, \dots, D_t\}$ onde cada dimensão D possuem um conjunto de valores $D = \{d_1, d_2, \dots, d_f\}$. Seja j o número de sessões multidimensionais $S' = \{s'_1, s'_2, \dots, s'_j\}$ onde cada sessão s' é uma tupla definida por um usuário $u \in U$, um conjunto de itens acessados $I_{s'} \subseteq I$ e um conjunto de dimensões $D_{s'} \subseteq D_1 \cup D_2 \cup \dots \cup D_t$ contendo todos os valores de dimensão associados com a sessão s' . A abordagem DaVI consiste em converter cada sessão multidimensional $s' = \langle u, I_{s'}, D_{s'} \rangle$ em uma sessão bidimensional $s'' = \langle u, I_{s''} \cup D_{s''} \rangle$ onde os valores das dimensões em $D_{s''}$ são usados como itens virtuais junto com os itens comuns em $I_{s''}$.

Uma grande vantagem desta abordagem, descrita em Domingues, Jorge e Soares (2012), é poder ser utilizada com diferentes algoritmos tradicionais, usando dados e obtendo modelos multidimensionais sem alterar esses algoritmos.

No Capítulo 5, são descritos os algoritmos de agrupamento que foram utilizados durante o desenvolvimento do trabalho.

Parte II

Algoritmos Utilizados

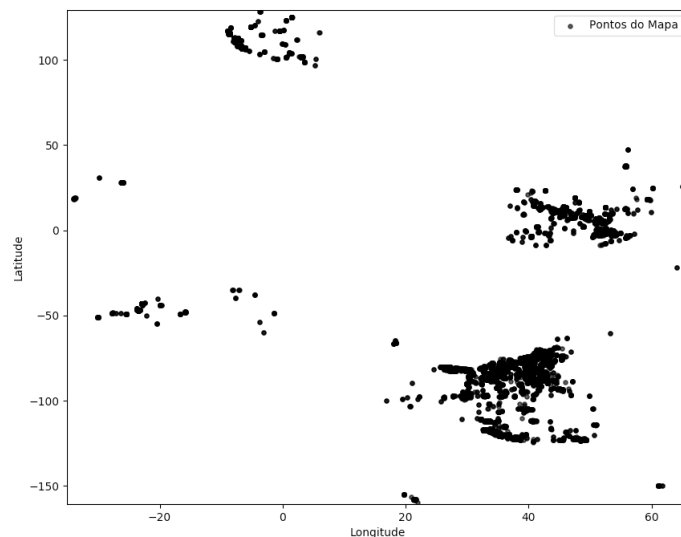
4 Algoritmos de Agrupamento

Neste capítulo serão apresentados os algoritmos de agrupamento que foram utilizados para o desenvolvimento do trabalho.

4.1 Algoritmo DBSCAN

Proposto inicialmente por Ester et al. (1996), o algoritmo DBSCAN (*Density Based Spatial Clustering of Applications with Noise*) pode ser utilizado para descobrir agrupamentos de dados a partir de uma visão espacial do banco de dados, utilizando atributos georreferenciados ou inteiros.

Figura 5 – Exemplos da visão do banco de dados por meio de gráfico



Um agrupamento de dados é reconhecido quando visualizado por meio de gráficos, a partir da densidade dos dados em uma determinada região, como visto na Figura 5. Uma das qualidades deste algoritmo é a habilidade de encontrar *clusters* que possuem qualquer tipo de formato, já que o formato do *cluster* em um banco de dados espacial pode ser alongado, esférico, ou de qualquer outra forma. (ESTER et al., 1996).

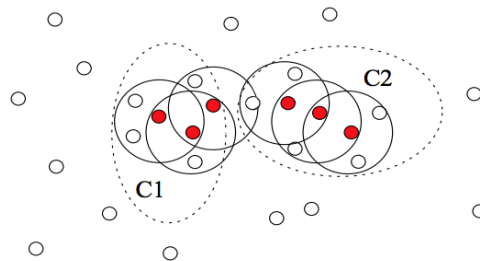
O algoritmo DBSCAN possui dois parâmetros para sua utilização: ε e *MinPts*, onde ε será o raio no qual a vizinhança de um ponto será definida e *MinPts* é o número de pontos que são necessários dentro da vizinhança- ε de um ponto para que ele esteja em um *cluster*. Em Ester et al. (1996), são definidos dois tipos de ponto, que são os **pontos de núcleo** e os **pontos de borda**. Os pontos de núcleo ficam no interior do *cluster* e geralmente possuem uma vizinhança- ε significativamente maior que os pontos de borda, que ficam na borda do *cluster*.

O *cluster* formado pelo DBSCAN, como apresentado em [Han, Kamber e Tung \(2001\)](#), deve obedecer as seguintes regras:

1. Um ponto só pode estar em um *cluster* se, e somente se, estiver na vizinhança- ε de um ponto de núcleo de um *cluster*;
2. Um ponto de núcleo o dentro da vizinhança- ε de um ponto de núcleo p deve estar no mesmo *cluster* que p ;
3. Um ponto de borda q que está na vizinhança- ε de pontos de núcleo $p_1, \dots, p_i$ deve pertencer ao mesmo *cluster* que algum ponto de núcleo de $p_1, \dots, p_i$;
4. Um ponto de borda r que não está na vizinhança- ε de nenhum ponto de núcleo é considerado barulho.

Um exemplo apresentado na [Figura 6](#), demonstra o funcionamento do DBSCAN onde ε é assumido como o raio dos círculos e $MinPts = 3$. Os pontos de núcleo estão representados pelos círculos vermelhos e os pontos de borda como círculos brancos com a borda preta.

Figura 6 – Exemplo do algoritmo DBSCAN



Fonte – Adaptado de [Han, Kamber e Tung \(2001\)](#)

4.2 Algoritmo KMeans

De acordo com [Na, Xumin e Yong \(2010\)](#), o algoritmo KMeans foi apresentado em 1967, sendo ele um dos algoritmos mais simples para aprendizado não supervisionado. O KMeans é o algoritmo particional de agrupamento mais conhecido e mais utilizado devido a sua simplicidade e eficiência. O algoritmo KMeans tem como parâmetro um valor k , que representa o número de *clusters* que serão gerados. Dado um conjunto de pontos, o algoritmo então iterativamente particiona o conjunto de dados em k *clusters* de acordo com uma função de distância ([LIU, 2011](#)).

Em [Na, Xumin e Yong \(2010\)](#) é apresentado o algoritmo como um processo iterativo de duas fases separadas, onde a primeira fase é responsável por selecionar k pontos

centrais, chamados de **centroides**. Na segunda fase, os demais pontos da base de dados serão atribuídos ao centroide mais próximo.

Para definir os novos centroides em cada execução das fases, uma das medidas utilizadas é a média de cada ponto do *cluster*. Como o algoritmo é iterativo, as duas fases são executadas até que um critério de parada seja obtido, que são definidos em Liu (2011) como os seguintes:

1. Não existir reatribuições de pontos em *clusters* diferentes;
2. Não existir mudança de centroides;
3. Diminuição mínima na soma de quadrados residuais(SQR),

$$SQR = \sum_{j=1}^k \sum_{x \in C_j} dist(x, m_j)^2,$$

onde k corresponde ao número de *clusters*, C_j é o *cluster* número j , m_j sendo o centroide do *cluster* j e $dist(x, m_j)$ sendo a distância entre o ponto x e o centroide m_j .

Em Liu (2011) é apresentado um exemplo do funcionamento do algoritmo KMeans, onde existe um conjunto de dados em um espaço bidimensional e o número de *clusters* desejado corresponde a $k = 2$. Na Figura 7a, são selecionados dois pontos aleatórios da base de dados, que serão utilizados como centroides e serão alterados de acordo com a execução do algoritmo. A seguir, são demonstradas as iterações feitas pelo KMeans:

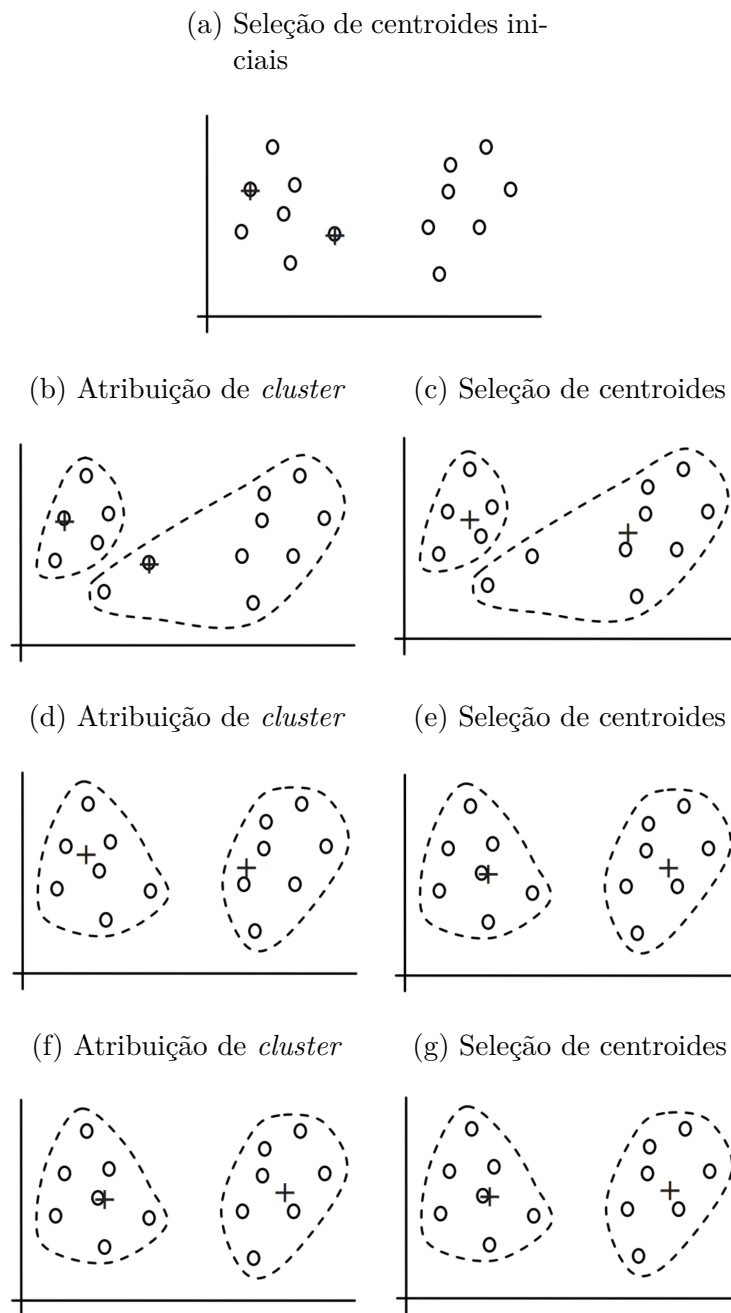
Iteração 01: Cada ponto é atribuído ao centroide mais próximo, como visto na Figura 7b. Na Figura 7c, é possível perceber que os novos centroides estão longe dos centroides iniciais. Esse comportamento demonstra a importância do processo iterativo;

Iteração 02: Novamente os pontos são atribuídos ao centroide mais próximo, como visto na Figura 7d, e então na Figura 7e os centroides são recalculados;

Iteração 03: Durante essa iteração, não ocorre uma atribuição de pontos diferente da iteração anterior, como visto na Figura 7f, não sendo necessário então a alteração dos centroides.

Os clusters finais obtidos no algoritmo KMeans estão na Figura 7g, que serão a saída do algoritmo KMeans.

Figura 7 – Funcionamento do algoritmo KMeans



Fonte – Adaptada de [Liu \(2011\)](#)

Em [Han, Kamber e Pei \(2012\)](#), algumas desvantagens do algoritmo KMeans são citadas. Uma das principais desvantagens do KMeans é a necessidade de existir uma média para poder fazer o cálculo do centroide, o que pode não existir em conjuntos de dados onde os atributos são nominais.

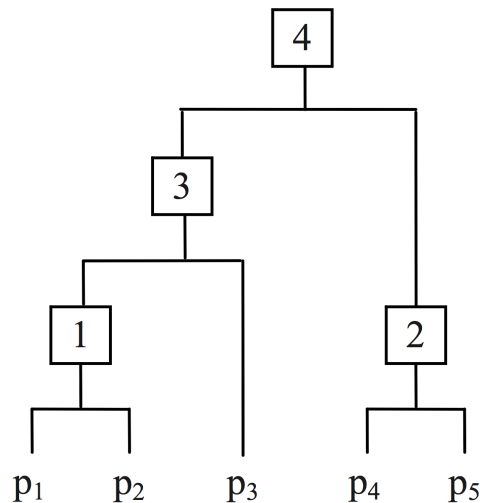
O algoritmo KMeans sofre uma perda de qualidade quando existem pontos chamados de **outliers**, que são pontos distantes de qualquer agrupamento de dados. Isso ocorre devido à utilização da média dos pontos para calcular o centroide e, quando existem pontos distantes, o resultado da média sofre interferência, atrapalhando então a atribuição

de pontos para *clusters* (HAN; KAMBER; PEI, 2012).

4.3 Algoritmo Aglomerativo

Dentro da classe de algoritmos de agrupamento hierárquicos, o algoritmo aglomerativo se encontra como o mais popular. O processo de agrupamento ocorre de modo que uma sequência aninhada de *clusters* (similar a uma árvore) seja construída. O algoritmo aglomerativo constrói essa árvore partindo das folhas (pontos do banco de dados), combinando pares de *clusters* que são similares (ou com menor distância), repetindo esse processo até que todos os pontos estejam em um único *cluster* (LIU, 2011). A Figura 8 apresenta uma árvore formada pelo algoritmo aglomerativo.

Figura 8 – Exemplo do algoritmo aglomerativo



Fonte – Liu (2011)

Para medir a distância entre *clusters* que podem ser combinados, foi utilizado o método de ligação médio, que corresponde a média da distância entre todos os pontos do *cluster* A para todos os pontos do *cluster* B (LIU, 2011).

Em Rokach e Maimon (2005), uma vantagem atribuída ao algoritmo aglomerativo se dá na utilização de uma árvore para representar os *clusters*, permitindo a escolha do número de partições de acordo com o nível de similaridade desejado.

O algoritmo aglomerativo não é recomendado para um conjunto de dados grandes, visto que a complexidade do mesmo é no mínimo $O(m^2)$ (onde m é o número de pontos), tornando-o não linear de acordo com o número de objetos (ROKACH; MAIMON, 2005).

5 Algoritmos de Recomendação

Neste capítulo são apresentados os algoritmos utilizados durante o trabalho, sendo que os dois primeiros algoritmos são algoritmos tradicionais de recomendação, enquanto o algoritmo apresentado na [seção 5.3](#) se utiliza de uma abordagem sensível ao contexto.

5.1 Algoritmo ItemKNN

O Algoritmo ItemKNN, conhecido como *Item-Based Collaborative Filtering*, foi apresentando em [Sarwar et al. \(2001\)](#), visando resolver os problemas observados nos algoritmos de filtragem colaborativa baseados em usuários, como por exemplo a escalabilidade, visto que o número de usuários tende a possuir um maior crescimento que o número de itens em uma aplicação.

A abordagem baseada em itens utiliza os itens que o usuário avaliou previamente e faz o cálculo da similaridade desses itens para com um item alvo i , selecionando então os k itens mais similares $\{i_1, i_2, \dots, i_k\}$. Ao mesmo tempo, suas similaridades são calculadas $\{s_{i_1}, s_{i_2}, \dots, s_{i_k}\}$ e, a partir do momento em que os itens mais similares são encontrados, a predição de itens não recomendados é calculada como a média ponderada das avaliações do usuário nestes itens similares ([SARWAR et al., 2001](#)).

Para calcular a similaridade entre os itens, foi utilizada a similaridade baseada em cosseno. Em [Sarwar et al. \(2001\)](#), a similaridade de dois itens A e B é apresentada da seguinte maneira:

$$similaridade(\vec{A}, \vec{B}) = \text{cosseno}(\vec{A}, \vec{B}) = \frac{\vec{A} \bullet \vec{B}}{\|\vec{A}\| * \|\vec{B}\|}$$

onde $\vec{A} \bullet \vec{B}$ denota o produto dos vetores, que pode ser visto como o somatório da multiplicação de todos os itens dos vetores e $\|\vec{A}\| * \|\vec{B}\|$ compreende a multiplicação do tamanho dos vetores.

Para fazer a predição de itens para o usuário, [Sarwar et al. \(2001\)](#) calcula a predição de um item i para um usuário u calculando a soma das avaliações dadas pelo usuário nos itens similares ao item i . Para cada avaliação, é aplicado um peso correspondente a similaridade $s_{i,j}$ entre os itens i e j . Os itens similares são denotados por N , e a predição pode ser denotada por:

$$P_{u,i} = \frac{\sum N(S_{i,N} * R_{u,N})}{\sum N(| S_{i,N} |)}$$

Esta abordagem tenta capturar como um usuário ativo avalia os itens similares, sendo que a soma ponderada escala de acordo com a soma dos itens similares para garantir que a predição está dentro do intervalo predefinido (SARWAR et al., 2001).

Os itens que possuem uma melhor predição e que ainda não foram avaliados por usuário, serão então organizados em uma lista ordenada de itens, sendo que os itens que estão nas primeiras posições da lista são os que obtiveram um valor de predição maior.

5.2 Algoritmo BPR

O algoritmo Bayesian Personalized Ranking (BPR), proposto em Rendle et al. (2012), consiste em duas partes: um critério de otimização BPR-Opt junto com um algoritmo genérico de aprendizado LearnBPR, que é baseado na descida de gradiente estocástico com amostragem em *bootstrap* de triplas de treino.

O critério de otimização BPR-Opt é um método genérico que resolve a tarefa de ranking personalizado, que consiste em apresentar ao usuário uma lista ordenada de itens por meio de uma análise Bayesiana do problema. Para encontrar uma lista personalizada de todos os itens $i \in I$ para o usuário, a fórmula Bayesiana maximiza a probabilidade posterior seguinte Θ que representa o vetor de parâmetros de um modelo abstrato.

Algoritmo 1: Algoritmo LearnBPR. Fonte: (RENDLE et al., 2012)

Data: D_s, Θ

1 inicializa Θ

2 **repeat**

3 seleciona (u, i, j) de D_s

4 $\Theta \leftarrow \Theta + \alpha \left(\frac{\epsilon^{-\hat{x}_{uij}}}{1 + \epsilon^{-\hat{x}_{uij}}} \cdot \frac{\delta}{\delta\Theta} + \lambda_{\Theta} \cdot \Theta \right)$

5 **until** convergência;

6 **return** $\hat{\Theta}$

O algoritmo LearnBPR, como apresentado no Algoritmo 1, possui como parâmetros D_s , que consiste na base de dados que será utilizada e Θ representa o vetor de parâmetros que será utilizado pelo algoritmo. Ele consiste de um laço principal que será executado até que se note uma convergência.

Dentro do laço principal do algoritmo, mais especificamente na linha 3, são escolhidos aleatoriamente 3 vetores de características: 2 vetores para itens (i e j) e 1 vetor para usuário (u).

Na linha 4 do algoritmo, o vetor de parâmetros é ajustado de acordo com a taxa de aprendizagem α , a taxa de regularização λ_{Θ} (que evita o *overfitting*) e com o critério de otimização BPR-Opt, que se utiliza da função que representa a função sigmoide $\epsilon^{-\hat{x}_{uij}}$, responsável por capturar a preferência do usuário u em um item i sobre um item j que

foram escolhidos no passo anterior.

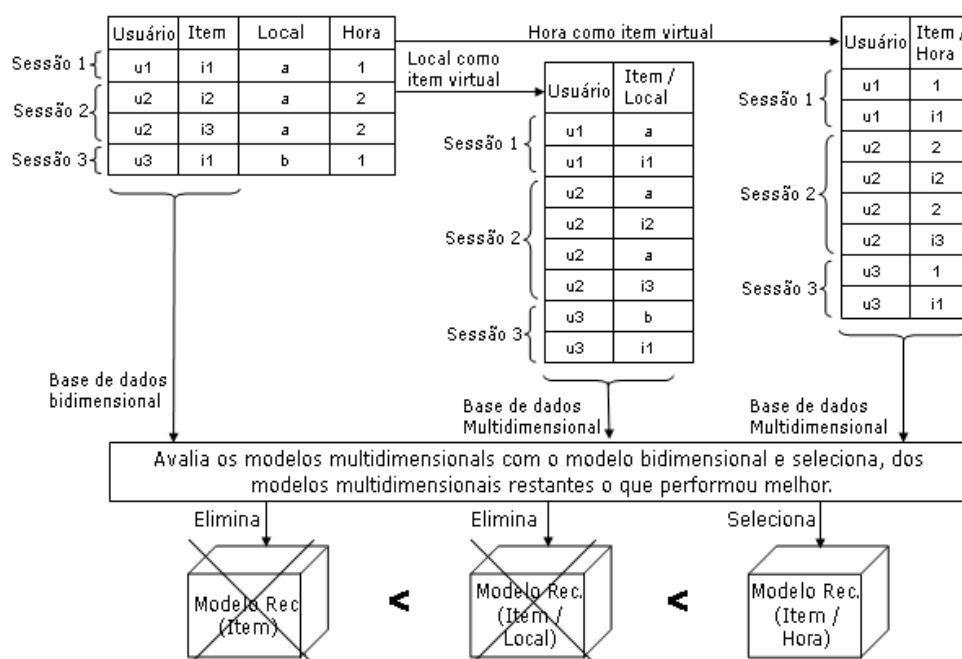
Com isso, o algoritmo LearnBPR definido em [Rendle et al. \(2012\)](#) consegue captar as preferências de todos os usuários sobre os itens e, a partir dessa informação, consegue fornecer uma lista ordenada de recomendações para o usuário.

5.3 Algoritmo DaVI-Best

A abordagem DaVI, apresentada na [seção 3.4](#), se utiliza das informações contextuais para criar itens virtuais. Uma preocupação das abordagens DaVI é determinar quais dimensões serão incluídas no modelo de recomendação, sabendo que algumas dessas dimensões possuem mais informação que outras ([DOMINGUES; JORGE; SOARES, 2012](#)).

Como apresentado em [Domingues, Jorge e Soares \(2012\)](#), o algoritmo DaVI-Best funciona de forma com que a melhor dimensão seja selecionada como item virtual para ser utilizada em um algoritmo de recomendação tradicional, como por exemplo ItemKNN e BPR. Em seu primeiro passo, cada dimensão em potencial é utilizada para construir um modelo de recomendação multidimensional utilizando a abordagem DaVI. Após isso, os modelos são avaliados e a melhor dimensão é selecionada, de acordo com alguma métrica escolhida. O conjunto de dados originais são então divididos em duas partes: uma que é utilizada para aprender o modelo e outra para avaliação, chamada de conjunto de validação.

Figura 9 – Exemplo de utilização do algoritmo DaVI-Best



Fonte – Adaptado de [Domingues \(2011\)](#)

Na [Figura 9](#), é apresentado o exemplo de uma base de dados que contém 2 dimensões: Local e Hora. São criados os modelos multidimensionais utilizando os itens virtuais relativos as dimensões e, após isso, são avaliados os modelos criados junto com o modelo tradicional. O que obteve a melhor performance, sendo no caso da figura o que utilizava as informações contextuais da dimensão Hora, é então selecionado e utilizado para fazer as recomendações finais para o usuário.

Parte III

Trabalho desenvolvido

6 Mineração de Dados

Neste capítulo serão descritos os trabalhos realizados tendo como base as etapas de mineração de dados apresentadas no [Capítulo 1](#). A etapa de pós-processamento não foi necessária no trabalho, visto que o *framework* utilizado para a extração de padrões já removia os dados que não foram atribuídos em alguma região.

6.1 Conhecimento do Domínio

Para que um sistema de recomendação sensível ao contexto seja utilizado ao invés de um sistema de recomendação tradicional, vê-se necessário a obtenção de informações contextuais que se sejam relevantes durante a utilização de um sistema de recomendação.

Nesse trabalho, o processo de mineração de dados foi guiado com o objetivo de obter informações contextuais a partir de informações georreferenciadas. Essas informações estavam na forma de latitude e longitude que, com o propósito de obter informações mais relevantes, foram agrupados em regiões geográficas.

6.2 Pré-processamento

Nesta seção serão apresentadas as bases de dados georreferenciadas que foram utilizadas para gerar os *clusters* para a obtenção de informação contextual junto com as tarefas de pré-processamento realizadas nos mesmos.

6.2.1 Foursquare

O Foursquare¹ é um serviço que fornece informações sobre atrações e restaurantes de acordo com a localização dada pelo usuário. Por meio das aplicações *Web* e *Mobile*, o Foursquare possui mais de 50 milhões de acessos mensais, possuindo mais de 12 bilhões de *check-ins*.

Quando um usuário deseja compartilhar em suas redes sociais que está em um estabelecimento, ele realiza um *check-in* no Foursquare. Essas informações de *check-in* foram utilizadas durante o desenvolvimento do trabalho para fazer recomendações.

A base de dados foi obtida em [Gao e Liu \(2014\)](#), e possui o histórico de *check-ins* de 11.326 usuários em um intervalo de tempo de Janeiro de 2011 até Dezembro de 2011, totalizando 1.385.223 *check-ins*.

¹ <https://foursquare.com/about>

6.2.2 Yelp

Assim como o Foursquare, Yelp² é um serviço que foi criado com a intenção de auxiliar seus usuários a encontrar comércio próximo a uma localização dada pelo usuário. São mais de 28 milhões de acessos mensais utilizando as aplicações *Web* e *Mobile* e 135 milhões de *reviews* escritos por usuários.

Os *reviews* consistem de duas partes: uma descrição textual do usuário sobre a experiência que teve no estabelecimento, no qual imagens podem ser anexadas, junto com uma avaliação que pode assumir valores entre 1 e 5. As informações de *reviews* foram utilizadas durante a etapa de extração de padrões para obter as regiões.

O Yelp disponibiliza³ um subconjunto de seus dados, que contém estabelecimentos, *reviews* e usuários de maneira gratuita para uso acadêmico. A base de dados utilizada contém 4.700.000 *reviews* efetuados por 1.100.000 usuários em 156.000 estabelecimentos.

6.2.3 Transformação dos Dados

Parte crucial de um processo de mineração de dados, o pré-processamento dos dados é essencial para obter um bom resultado final. Utilizando as duas bases de dados citadas na seção 6.2, foi realizada uma tarefa de pré-processamento com o objetivo de transformá-los em um padrão que fosse utilizado pelo resto do processo. Essa transformação dos dados consistiu de duas tarefas: remoção de atributos e remoção de *singletons*⁴.

A remoção de atributos consiste em retirar atributos que não serão utilizados durante o processo de agrupamento, deixando apenas atributos que são relevantes. Como este trabalho tem o foco em agrupar dados relacionados a geolocalização, atributos que podem ser considerados importantes são latitude e longitude, que são relacionados a coordenadas geográficas. Os demais atributos contextuais, utilizando como exemplo informações temporais como dia, mês e hora, não são necessários para o agrupamento, sendo então removidos.

A remoção de *singletons* consiste em remover todos os acessos únicos das bases de dados, sejam eles feitos por usuários, ou por itens que tenham sido avaliados apenas uma vez. Como visto em Domingues, Soares e Jorge (2008), o resultado da retirada de *singletons* em sistemas de recomendação possui um efeito benéfico para o resultado final. A Tabela 1 apresenta o formato das bases de dados que foi obtido após o pré-processamento.

² <https://www.yelp.com/about>

³ <https://www.yelp.com/dataset/download>

⁴ Uma única pessoa ou objeto de algum tipo sob consideração.

Tabela 1 – Exemplo dos atributos da base de dados.

Identificador usuário	Identificador item	Avaliação	Latitude	Longitude
774	6391	1	48.579718	7.746161
12021	1032	1	40.883266	-74.242431
16925	568	1	44.98186	-93.277445
9889	302	1	33.6775	-111.977
5417	528	1	42.349535	-71.040589

Uma outra tarefa de pré-processamento foi utilizada nas bases de dados em conjunto com a transformação dos dados. Foi realizada uma redução do número de entradas, devido ao grande custo computacional dos algoritmos utilizados no trabalho. Para a redução, foram escolhidos aleatoriamente dados do conjunto original que fizeram parte do subconjunto utilizado para o processo de agrupamento. As informações sobre as bases de dados, após o pré-processamento completo está na [Tabela 2](#).

Tabela 2 – Bases de dados após o pré-processamento

Dataset	Número de Usuários	Número de Itens	Número de Avaliações
Yelp	6.049	5.414	19.986
Foursquare	4.921	3.542	19.848

6.3 Extração de Padrões

A extração de padrões, como visto na [seção 1.3](#), tem como objetivo executar os algoritmos de mineração de dados. Nesse trabalho, foram utilizados os algoritmos apresentados no [Capítulo 4](#).

Para escolher os algoritmos que foram utilizados no trabalho, foi levado em consideração os tipos de algoritmos de extração de padrões existentes. Como nas técnicas de agrupamento os algoritmos são divididos em dois grupos principais, particionamento e hierárquico, foi escolhido um algoritmo de cada grupo.

O algoritmo DBSCAN foi escolhido, pois o mesmo se utiliza de densidade espacial para realizar a tarefa de agrupamento. Pelos dados possuírem atributos espaciais (latitude e longitude), a escolha de um algoritmo que utilizasse esses atributos é justificada.

Seguindo o objetivo do trabalho de utilizar informações georreferenciadas em sistemas de recomendação, a etapa de extração de padrões utilizou os dados de latitude e longitude referentes aos *reviews* e aos *checkins* como entrada para os algoritmos de agrupamento. A partir dessas informações, foram obtidos então *clusters*, que foram tratados como regiões, isto é, informação contextual, por se tratarem de coordenadas geográficas.

Na [Tabela 3](#), é apresentado um exemplo do resultado da execução de um algoritmo de agrupamento, com a atribuição dos itens em suas respectivas regiões.

Tabela 3 – Exemplo das regiões atribuídas para cada item.

Identificador usuário	Identificador item	Avaliação	Região
774	6391	1	região0
12021	1032	1	região1
16925	568	1	região1
9889	302	1	região1
5417	528	1	região4

Para executar os algoritmos de agrupamento, foi utilizada o *framework scikit-learn* (PEDREGOSA et al., 2011), uma ferramenta construída na linguagem de programação *Python*. No *scikit-learn*, existem diversos algoritmos que podem ser utilizados durante a etapa de extração de padrões além dos algoritmos de agrupamento, como algoritmos de classificação e redes neurais.

6.4 Utilização do Conhecimento

Cada entrada da base de dados, após o processo de extração de padrões, teve uma região geográfica atribuída. Esse conhecimento foi utilizado como contexto para sistemas de recomendação, visando obter melhores resultados do que um sistema de recomendação tradicional. Nesse trabalho, o algoritmo DaVI-Best foi utilizado em conjunto com o ItemKNN representando um algoritmo de recomendação sensível ao contexto, e foi comparado com o ItemKNN tradicional. O mesmo foi feito com o algoritmo BPR.

Para avaliar se as recomendações geradas utilizando as regiões são melhores, foram utilizados os algoritmos descritos no Capítulo 5. Foram utilizadas também as métricas *Precision*, *Recall* e *F-measure*.

Em Shani e Gunawardana (2011), a métrica *Precision* é definida como o número de itens recomendados que são relevantes, enquanto *Recall* é definida como o número de de itens relevantes que são recomendados. Para obter essas métricas, foi utilizado H como sendo o conjunto dos itens que foram escondidos e R como sendo o conjunto de recomendações que o sistema de recomendação fez (DOMINGUES; JORGE; SOARES, 2012). As fórmulas para as métricas são as seguintes:

$$Precision = \frac{|R \cap H|}{|R|}$$

$$Recall = \frac{|R \cap H|}{|H|}$$

A métrica F1 pode ser interpretada como a média harmônica entre os valores de *Precision* e *Recall*, possuindo uma variação entre 0 a 1, sendo 0 o menor valor e 1 o maior. Sua fórmula é apresentada da seguinte forma

$$\text{F-measure} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}.$$

A execução dos algoritmos de recomendação junto com o cálculo das métricas se deu por meio da utilização do CARSKit (ZHENG; MOBASHER; BURKE, 2015), um *framework* de código-livre escrito em *Java*, projetado para sistemas de recomendação sensíveis ao contexto que contém algoritmos que são considerados estado da arte. Ele é uma extensão de um *framework* chamado LibRec⁵, que têm foco em sistemas de recomendação tradicionais.

⁵ <https://www.librec.net>

7 Resultados obtidos

Neste capítulo, serão apresentados os resultados obtidos com os experimentos realizados. Na [seção 7.1](#) e [seção 7.2](#), serão apresentados os resultados tendo como base os sistemas de recomendação utilizados para a avaliação. Na [seção 7.3](#), serão apresentados os resultados de uma maneira mais ampla, comparando os melhores resultados obtidos com ambos os algoritmos.

7.1 ItemKNN

Nesta seção, serão apresentados os resultados obtidos com os algoritmos de agrupamento (com a informação contextual) em comparação com o algoritmo de *baseline* (sem informação contextual). O algoritmo DaVI-Best, utilizando o sistema de recomendação ItemKNN como sistema de recomendação tradicional, foi utilizado com as informações contextuais, enquanto o algoritmo de baseline foi o algoritmo ItemKNN tradicional.

Ao utilizar a base de dados do Foursquare, que contém informações de *check-in*, obtemos uma melhora na métrica F-measure para os três algoritmos de agrupamento utilizados. A escolha dos parâmetros dos algoritmos de agrupamento foram feitas de modo que houvesse uma variação do número de *clusters*, com o objetivo de descobrir algum padrão relativo a quantidade de clusters e o resultado obtido pelas métricas. Na [Tabela 4](#), foram destacados os melhores resultados obtidos para cada algoritmo de agrupamento.

Tabela 4 – Resultados obtidos com o ItemKNN no Foursquare

Algoritmo de cluster	Num. clusters	Precision	Recall	F-measure
Baseline		0,000873	0,006676	0,001518
Aglomerativo	2	0,000873	0,006676	0,001518
Aglomerativo	4	0,000884	0,006734	0,001538
Aglomerativo	8	0,000867	0,006724	0,001512
Aglomerativo	12	0,000867	0,006724	0,001512
Aglomerativo	16	0,000867	0,006724	0,001512
DBSCAN	7	0,000872	0,006661	0,001519
DBSCAN	12	0,000884	0,006734	0,001538
DBSCAN	16	0,000884	0,006734	0,001538
DBSCAN	19	0,000884	0,006734	0,001538
DBSCAN	26	0,000884	0,006734	0,001538
KMeans	3	0,000884	0,006734	0,001538
KMeans	5	0,000867	0,006724	0,001512
KMeans	10	0,000857	0,007316	0,001517

Os resultados obtidos com o ItemKNN para a base do Foursquare indicam que

um número menor de *clusters* conseguiu obter uma melhora. Apesar da melhora obtida pelos algoritmos ser de apenas 1,31% na *F-measure* para o *baseline*, pode-se destacar que a utilização das informações de check-in pode ajudar um usuário a obter recomendações melhores.

Utilizando os dados do Yelp, que contém informações de *reviews* feitos pelos usuários, foi obtida também uma melhora nos três algoritmos com relação ao *baseline*, porém com os algoritmos KMeans e Aglomerativo alcançando valores mais altos que o DBSCAN. A Tabela 5 apresenta os resultados obtidos com a base de dados Yelp.

Tabela 5 – Resultados obtidos com o ItemKNN no Yelp

Algoritmo de cluster	Num. clusters	Precision	Recall	F-measure
Baseline		0,003598	0,013619	0,005446
Aglomerativo	2	0,003288	0,023944	0,005565
Aglomerativo	4	0,003288	0,023944	0,005565
Aglomerativo	8	0,003265	0,023937	0,005534
Aglomerativo	16	0,003265	0,023937	0,005534
DBSCAN	8	0,003110	0,023861	0,005333
DBSCAN	13	0,003110	0,023861	0,005333
DBSCAN	23	0,003261	0,023933	0,005530
DBSCAN	53	0,003261	0,023933	0,005530
KMeans	2	0,003288	0,023944	0,005565
KMeans	4	0,003288	0,023944	0,005565
KMeans	8	0,003265	0,023937	0,005534
KMeans	16	0,003187	0,023884	0,005426

Apesar dos resultados da métrica *Precision* serem maiores para o *baseline*, as métricas *Recall* e *F-measure* obtiveram uma melhora nos demais algoritmos. Isso pode evidenciar que as recomendações feitas utilizando as informações contextuais de região são melhores que as recomendações que não se utilizam dessa informação, apesar do número de itens recomendados que são relevantes do algoritmo de *baseline* ser maior, como garantido pela métrica *Precision*, as recomendações em geral, foram melhores utilizando as informações de região.

A melhora da *F-measure* dos algoritmos Aglomerativo e KMeans com relação ao *baseline* foi de 2,18%, enquanto a melhora do algoritmo DBSCAN foi de apenas 1,54%.

7.2 BPR

O algoritmo BPR, junto com a utilização do DaVI-Best, produziram os melhores resultados obtidos nesse experimento. O BPR foi utilizado como sistema de recomendação tradicional com o DaVI-Best e como *baseline* para comparar os resultados obtidos com informação contextual e sem essa informação.

Como apresentado na [Tabela 6](#), o ganho obtido em todas as métricas utilizando a base de dados do Foursquare foi significativo. Entre os algoritmos, o DBSCAN performou melhor que os demais, alcançando uma melhora de 189,67 % na métrica *F-measure* com relação ao *baseline*. A melhora obtida pelos demais algoritmos na métrica *F-measure* ainda assim foi expressiva, com 178,06% para o algoritmo Aglomerativo e 170,49% para o KMeans.

Tabela 6 – Resultados obtidos com o BPR no Foursquare

Algoritmo de cluster	Num. cluster	Precision	Recall	F-measure
Baseline		0,002319	0,016333	0,003874
Aglomerativo	2	0,005544	0,044737	0,009704
Aglomerativo	4	0,006157	0,049825	0,010772
Aglomerativo	8	0,005946	0,046994	0,010376
Aglomerativo	12	0,006079	0,047969	0,010601
Aglomerativo	16	0,005589	0,043755	0,009742
DBSCAN	7	0,005394	0,042350	0,009408
DBSCAN	12	0,005852	0,048167	0,010281
DBSCAN	16	0,006382	0,052618	0,011222
DBSCAN	19	0,005243	0,042015	0,009167
DBSCAN	26	0,005622	0,045710	0,009859
KMeans	3	0,005907	0,046457	0,010282
KMeans	5	0,005970	0,048917	0,010479
KMeans	10	0,005643	0,046180	0,009912

Apesar da métrica *Precision* ter um valor baixo, a *Recall* obter um valor alto mostra que grande parte das recomendações relevantes foram realizadas.

Tabela 7 – Resultados obtidos com o BPR no Yelp

Algoritmo de cluster	Num. cluster	Precision	Recall	F-measure
Baseline		0,002875	0,020781	0,004855
Aglomerativo	2	0,003065	0,022937	0,005221
Aglomerativo	4	0,003054	0,022941	0,005201
Aglomerativo	8	0,002941	0,022375	0,005045
Aglomerativo	16	0,002749	0,020455	0,004705
DBSCAN	8	0,003199	0,024231	0,005484
DBSCAN	13	0,003361	0,026207	0,005795
DBSCAN	23	0,002869	0,022062	0,004919
DBSCAN	53	0,003042	0,022817	0,005194
KMeans	2	0,002900	0,021289	0,004943
KMeans	4	0,003080	0,022841	0,005239
KMeans	8	0,003390	0,025482	0,005785
KMeans	16	0,003166	0,024374	0,005469

Os resultados obtidos com o BPR utilizando a base de dados do Yelp apresentaram uma melhora com relação ao algoritmo tradicional, como visto na [Tabela 7](#). Apesar dos resultados não serem tão expressivos quanto os do Foursquare, todas as métricas obtiveram

melhora, com destaque para a métrica *F-measure*. O algoritmo com melhor desempenho foi o DBSCAN, que atingiu uma melhora de 19,36% com relação ao *baseline*, enquanto o KMeans obteve 19,15% e o Aglomerativo 7,53%.

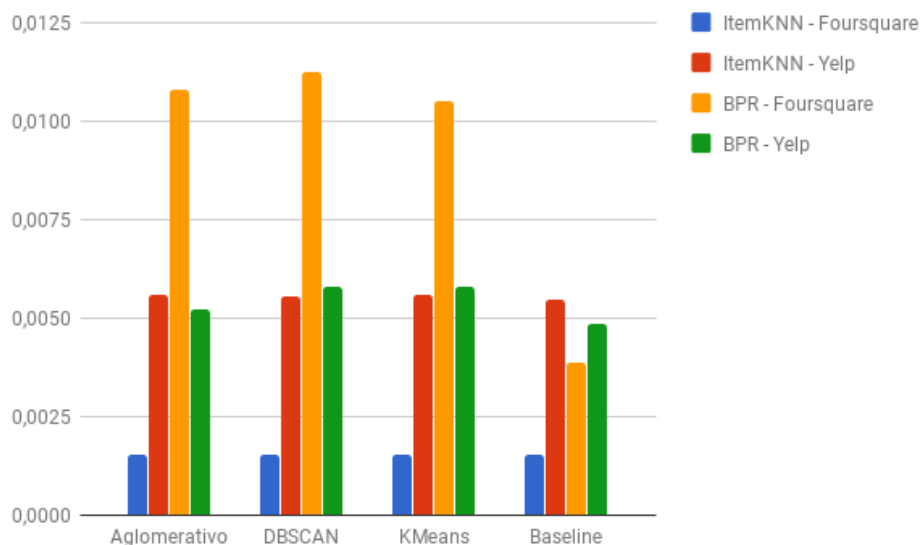
7.3 Discussão dos resultados

Tendo como base os resultados apresentados na seção anterior, pode-se concluir que o algoritmo ItemKNN, apesar de melhorar as recomendações se utilizando das informações das regiões, não é possível concluir qual dos algoritmos de agrupamento produz o melhor resultado, visto que para a base de dados do Foursquare, os valores da métrica *F-measure* foram iguais. Já nos resultados do Yelp, os resultados do algoritmo aglomerativo e do KMeans apesar de serem iguais, foram superiores ao do DBSCAN.

Utilizando o algoritmo BPR, os resultados do DBSCAN se provaram superiores ao dos demais algoritmos, em ambas as bases de dados. Foi possível observar também que houve uma superioridade do algoritmo BPR com relação ao ItemKNN, com resultados muito mais expressivos.

Na [Figura 10](#), podemos perceber que o melhor resultado obtido no experimento foi utilizando o DaVI-Best com o sistema de recomendação tradicional BPR, junto com o algoritmo de agrupamento DBSCAN. O DBSCAN foi o único algoritmo utilizado em que o número de *clusters* não era parâmetro de entrada e, portanto, viu-se necessário realizar alguns testes com o objetivo de obter uma varida quantidade de *clusters*. Os parâmetros utilizados no DBSCAN que produziram o melhor resultado, na base de dados Foursquare, foram $\varepsilon = 0.1$ e $MinPts = 1$.

Figura 10 – F-measure dos algoritmos de agrupamento



CONCLUSÃO

Neste trabalho foi avaliada a importância do uso de informações georreferenciadas em sistemas de recomendação sensíveis ao contexto. Para obter essa informação, foi utilizado um processo de mineração de dados, empregando técnicas de agrupamento presentes na literatura com o objetivo de obter regiões como informação contextual.

Foi feita uma análise de três algoritmos de agrupamento, visando identificar qual algoritmo resultaria em um resultado melhor ao ser utilizado em conjunto com um sistema de recomendação sensível ao contexto. Além dessa análise, foram testados dois sistemas de recomendações tradicionais que seriam usados com a abordagem DaVI: ItemKNN e BPR.

A melhor combinação de algoritmos, para ambas as bases de dados utilizadas no trabalho, foi obtida com a abordagem DaVI se utilizando do sistema de recomendação BPR, com as informações das regiões obtidas pelo DBSCAN. Foi observada uma melhora de 189,67% na métrica *F-measure* com relação ao algoritmo de *baseline*.

A partir do resultado obtido, é possível concluir que os objetivos propostos para esse trabalho foram alcançados, visto que foi observada uma melhora no resultado final.

Com isso, informações georreferenciadas podem ser utilizadas como contexto em sistemas de recomendação sensíveis ao contexto, e a utilização de regiões como informação contextual ao invés da utilização de algoritmos tradicionais resulta em uma melhora.

Como trabalhos futuros, a utilização de outras técnicas de agrupamento podem ser avaliadas com o objetivo de obter regiões melhores, além de outras técnicas de aprendizagem de máquina. Além disso, pode ser feito uma análise que relacione a heurística de *Silhouette* (ROUSSEEUV, 1987), que tenta mensurar em um número real que varia de 0 a 1, a qualidade do *cluster* (no caso do trabalho, região) com a *F-measure*.

Referências

- ADOMAVICIUS, G. et al. Incorporating contextual information in recommender systems using a multidimensional approach. In: *Electronic Commerce Research*. [S.l.]: ACM Transactions on Information Systems, 2005. v. 23, p. 103–145.
- ADOMAVICIUS, G.; TUZHILIN, A. Extending recommender systems: A multidimensional approach. In: *IJCAI'01 Proceedings of the 17th international joint conference on Artificial intelligence*. Seattle, WA, USA: Morgan Kaufmann Publishers, 2001. v. 1.
- ADOMAVICIUS, G.; TUZHILIN, A. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, v. 17, n. 6, 2005.
- ADOMAVICIUS, G.; TUZHILIN, A. Context-aware recommender systems. In: RICCI, F. et al. (Ed.). *Recommender Systems Handbook*. 1. ed. [S.l.]: Springer US, 2011. cap. 7.
- ANSARI, A.; ESSEGAIER, S.; KOHLI, R. Internet recommendation systems. *Journal of Marketing Research*, v. 37, n. 3, p. 363 – 375, 2000.
- BERRY, A.; LINOFF, G. *Data Mining Techniques: For Marketing, Sales and Customer Support*. [S.l.]: Wiley Computer Publishing, 1997.
- BOBADILLA, J. et al. Recommender systems survey. *Knowledge-Based Systems*, v. 46, p. 109–132, 2013.
- BREESE, J.; HECKERMAN, D.; KADIE, C. Empirical analysis of predictive algorithms for collaborative filtering. In: COOPER, G.; MORAL, S. (Ed.). *UAI'98 Proceedings of the 14th Conference Conference on Uncertainty in Artificial Intelligence*. [S.l.]: Morgan Kaufmann, 1998.
- DOMINGUES, M. A. Exploiting multidimensional data for web site automation. *AI Commun.*, v. 24, n. 3, p. 289–290, 2011.
- DOMINGUES, M. A.; JORGE, A. M.; SOARES, C. Dimensions as virtual items: Improving the predictive ability of top-n recommender systems. *Information Processing and Management*, 2012.
- DOMINGUES, M. A.; SOARES, C.; JORGE, A. M. An empirical study on the impact of singleton web accesses on the accuracy of recommender systems. *SBIA'08 Workshop sobre Web and Text Intelligence*, 2008.
- ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: *Proceedings of the seconds international conference on Knowledge Discovery and Data Mining*. [S.l.]: AAAI Press, 1996. cap. Spatial, Text and Multimedia Data Mining.
- FAYYAD, U.; PIATETSKY-SHAPIRO, G.; SMYT, P. From data mining to knowledge discovery in databases. *Artificial Intelligence Magazine*, v. 17, n. 3, 1996.

- FAYYAD, U.; PIATETSKY-SHAPIO, G.; SMYTH, P. Advances in knowledge discovery and data mining. In: *AAAI'96 Proceedings of the 13th national conference on Artificial intelligence*. Portland, Oregon: AAAI Press, 1996.
- FLACH, P. A.; LAVRAC, N.; ZUPAN, B. *Decompositions, dependencies, and Bayesian networks*. [S.l.], 1999.
- GAO, H.; LIU, H. *Location-Based Social Network Data Repository*. 2014. Disponível em: <http://www.public.asu.edu/~hgao16/dataset.html>.
- HAN, J.; KAMBER, M. Cluster analysis. In: *Data Mining: Concepts and Techniques*. 2. ed. [S.l.]: Morgan Kaufmann Publishers, 2006, (The Morgan Kaufmann Series in Data Management Systems). cap. 7.
- HAN, J.; KAMBER, M.; PEI, J. Cluster analysis: Basic concepts and methods. In: *Data Mining: Concepts and Techniques*. 3. ed. [S.l.]: Morgan Kaufmann Publishers, 2012, (The Morgan Kaufmann Series in Data Management Systems). cap. 10.
- HAN, J.; KAMBER, M.; TUNG, A. K. H. Spatial clustering methods in data mining: A survey. In: MILLER, H. J.; HAN, J. (Ed.). *Geographic Data Mining and Knowledge Discovery, Research Monographs in GIS*. [S.l.]: Taylor and Francis, 2001.
- LIU, B. Unsupervised learning. In: LIU, B. (Ed.). *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. [S.l.]: Springer, 2011. cap. 4, p. 117 – 149.
- LIU, B.; HSU, W. Post analysis of learned rules. In: *AAAI'96 Proceedings of the 13th national conference on Artificial intelligence*. Portland, Oregon: AAAI Press, 1996.
- LOPS, P.; GEMMIS, M. de; SEMERARO, G. Content-based recommender systems: State of the art and trends. In: RICCI, F. et al. (Ed.). *Recommender Systems Handbook*. 1. ed. [S.l.]: Springer US, 2011. cap. 3.
- MAIMON, O.; ROKACH, L. Introduction to knowledge discovery and data mining. In: *Data Mining and Knowledge Discovery Handbook*. 2. ed. [S.l.]: Springer, 2010. cap. 1.
- MILLER, H. J. Geographic data mining and knowledge discovery. *The handbook of geographic information science*, Blackwell Publishing Ltd, p. 352–366, 2008.
- MOBASHER, B. Web usage mining. In: LIU, B. (Ed.). *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. 2. ed. [S.l.]: Springer, 2011. cap. 12, p. 527 – 540.
- NA, S.; XUMIN, L.; YONG, G. Research on k-means clustering algorithm: An improved k-means clustering algorithm. In: *Proceedings of the 2010 Third International Symposium on Intelligent Information Technology and Security Informatics*. Washington, DC, USA: IEEE Computer Society, 2010.
- PALMISANO, C.; TUZHILIN, A.; GORGOGLIONE, M. Using context to improve predictive modeling of customers in personalization applications. *IEEE Transactions on Knowledge and Data Engineering*, v. 20, n. 11, 2008.
- PAZZANI, M. J.; BILLSUS, D. Content-based recommendation systems. In: *The Adaptive Web*. [S.l.]: Springer, 2007, (Lecture Notes in Computer Science). cap. 10.

- PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- PIERRAKOS, D. et al. Web usage mining as a tool for personalization: A survey. *User Modeling and User-Adapted Interaction*, v. 13, n. 4, p. 311–372, Nov 2003.
- RENDLE, S. et al. Bpr: Bayesian personalized ranking from implicit feedback. *CoRR*, 2012.
- REZENDE, S. O. et al. Mineração de dados. In: REZENDE, S. O. (Ed.). *Sistemas Inteligentes - Fundamentos e Aplicações*. [S.l.]: Manole, 2003. cap. 12.
- RICCI, F.; ROKACH, L.; SHAPIRA, B. Introduction to recommender systems handbook. In: RICCI, F. et al. (Ed.). *Recommender Systems Handbook*. 1. ed. [S.l.]: Springer US, 2011.
- ROKACH, L. A survey of clustering algorithms. In: *Data Mining and Knowledge Discovery Handbook*. 2. ed. [S.l.]: Springer, 2010. cap. 14.
- ROKACH, L.; MAIMON, O. Clustering methods. In: *Data Mining and Knowledge Discovery Handbook*. 1. ed. [S.l.]: Springer-Verlag New York, Inc., 2005. cap. 15.
- ROUSSEEUW, P. J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, v. 20, n. Supplement C, p. 53 – 65, 1987.
- SARWAR, B. et al. Item-based collaborative filtering recommendation algorithms. In: ACM. *Proceedings of the 10th international conference on World Wide Web*. [S.l.], 2001.
- SHANI, G.; GUNAWARDANA, A. Evaluating recommendation systems. In: RICCI, F. et al. (Ed.). *Recommender Systems Handbook*. 1. ed. [S.l.: s.n.], 2011. cap. 8.
- WEISS, S. M.; INDURKHIA, N. *Predictive Data Mining: A Practical Guide*. [S.l.]: Morgan Kaufmann Publishers, 1998.
- ZHENG, Y.; MOBASHER, B.; BURKE, R. D. Carskit: A java-based context-aware recommendation engine. *IEEE International Conference on Data Mining Workshop (ICDMW)*, p. 1668–1671, 2015.